

YourCrawl — Product Manager Master Overview

System Architecture · Cross-team Dependencies · 8-Week Roadmap

Product Manager

YourCrawl Dark Pattern Auditor

Generated: 16 August 2026

YourCrawl — Product Manager Master Overview

Product: YourCrawl Dark Pattern Auditor

Status: Pre-Production (Functional Prototype)

Goal: Ship a production-ready, multi-user SaaS product

Product Vision

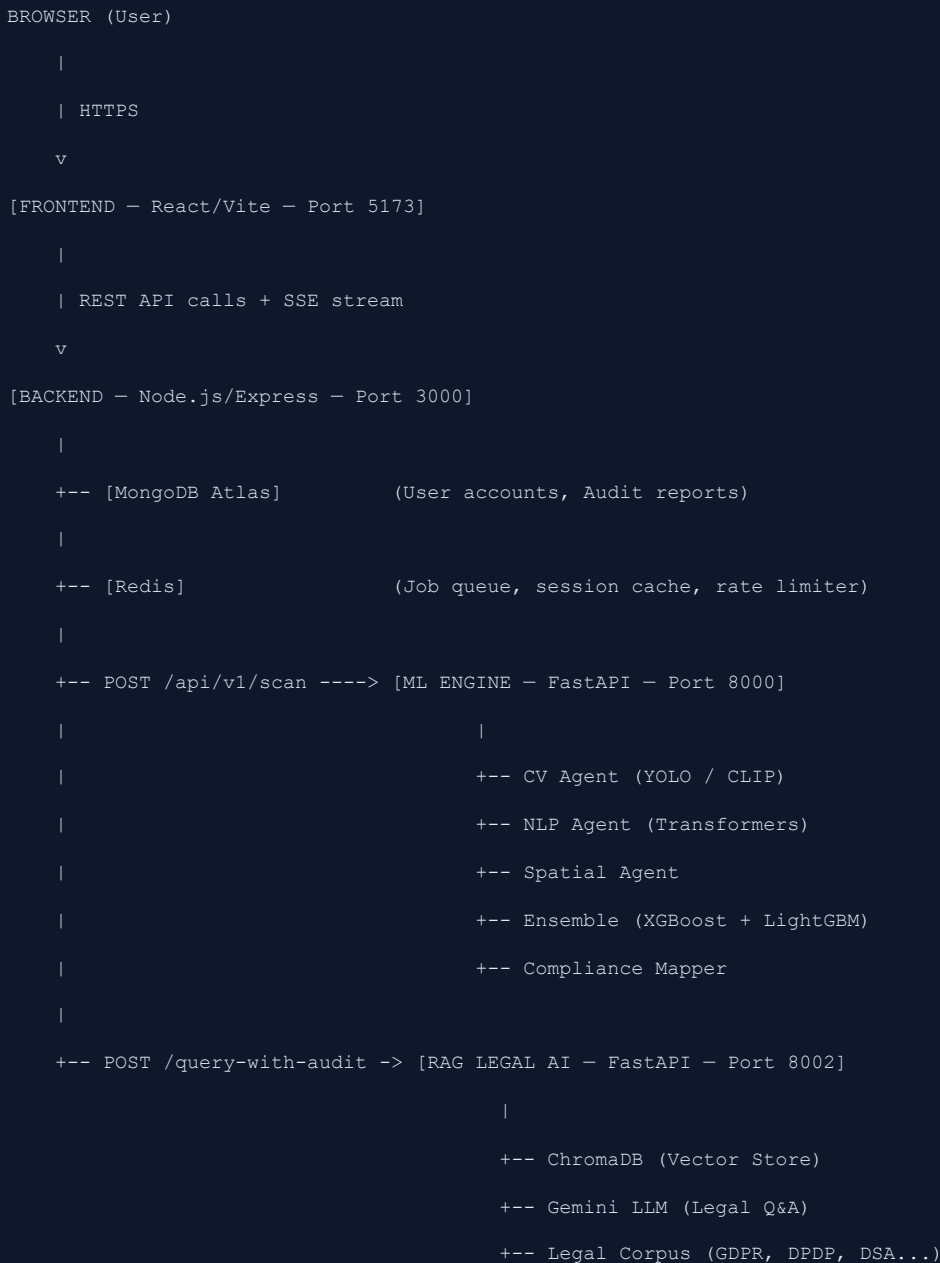
YourCrawl is a SaaS platform that automatically audits any website for dark patterns — deceptive UI/UX design tactics that manipulate users. It crawls a target website, uses computer vision and NLP to detect dark patterns, maps them to legal regulations (GDPR, DPDP Act, EU AI Act), and generates a prioritized remediation roadmap with an AI assistant for follow-up legal questions. **Core Value Proposition:** "Submit a URL. Get a full legal compliance audit in minutes." **Target Customers:** Product teams, UX teams, legal/compliance officers, digital agency QA departments, enterprise companies worried about EU AI Act / DPDP Act exposure.

Overall System Assessment

Service	Technology	Current Maturity	Production Readiness
Backend (Team 1)	Node.js + Express + Puppeteer	Prototype	25% — Synchronous pipeline is a showstopper
ML Engine (Team 2)	Python + FastAPI + XGBoost/CLIP	Advanced Prototype	45% — Architecture is strong but models may not be persisted
RAG Legal AI (Team 3)	Python + FastAPI + ChromaDB	Early Prototype	35% — Local DB, thin corpus, no conversation memory
Frontend (Team 4)	React + Vite + Tailwind	Shell/Prototype	30% — Mock data still used, no real-time UI

Estimated time to production-ready product with 4 focused developers: 6-8 weeks

Complete System Architecture



Cross-Team Dependency Map

Understanding which team blocks which is critical for sequencing work.

Dependency	From	To	Impact
------------	------	----	--------

Job Queue (Redis)	Backend Team 1	Frontend Team 4	Frontend cannot build real-time progress UI until Backend exposes SSE
Redis Scan Store	Backend Team 1	ML Team 2	ML needs Redis for persistent async scan storage
Redis Session Store	Backend Team 1	RAG Team 3	RAG needs Redis for conversation memory
ML API contract	ML Team 2	Backend Team 1	Backend must update to handle new ML response format if ML changes schemas
RAG streaming	RAG Team 3	Frontend Team 4	Frontend AI chat streaming depends on RAG SSE endpoint

Critical Path: Backend must complete Feature 1 (Job Queue) in Week 1 before Frontend can proceed with real-time UI.

Top 10 Showstopper Issues (Must Fix Before Launch)

1. Synchronous Audit Pipeline (Backend)

The HTTP request blocks for 2-3 minutes. Users will abandon, browser will timeout, and work is lost. Fix: Bull job queue + SSE. Team 1, Week 1.

2. Credentials Committed to Repository (Backend)

Real API keys, database passwords, and JWT secrets are visible in `Backend/.env`. This is a critical security vulnerability. Fix: Rotate all credentials NOW. Team 1, Immediately.

3. ML Models Not Persisted (ML)

If XGBoost/LightGBM models are trained on every startup (not loaded from disk), the service is 30-60 seconds slow to start and produces non-deterministic results. Fix: Verify, train once, save with joblib. Team 2, Week 1.

4. ChromaDB on Local Ephemeral Disk (RAG)

The vector database will lose all embeddings on any restart. Fix: Mount to persistent volume or migrate to hosted Chroma. Team 3, Week 1.

5. Frontend Still Uses Mock Data (Frontend)

Users see fake audit results. Fix: Remove all `mockData.js` imports. Team 4, Week 1.

6. No Rate Limiting (Backend)

A single bad actor can run unlimited free audits, drain Gemini API credits, and crash the server. Fix: express-rate-limit. Team 1, Week 2.

7. GDPR Not in Legal Corpus (RAG)

The ML service references GDPR violations, but the RAG agent cannot answer GDPR questions. This is a live inconsistency. Fix: Add GDPR document to corpus. Team 3, Week 1.

8. No Usage Metering (Backend)

Cannot monetize without usage limits. Fix: Plan tiers + monthly audit counter. Team 1, Week 2.

9. In-Memory Async Scan Store (ML)

Async ML scans are lost on restart. Fix: Redis-backed scan store. Team 2, Week 2.

10. No Conversation Memory in AI Assistant (RAG)

The AI assistant treats every question as isolated — cannot handle follow-ups. Fix: Redis session store for conversation history. Team 3, Week 2.

8-Week Production Roadmap (All Teams)

Week 1 – Foundation (Critical path items)

- └ Backend: Job Queue (Bull/Redis) + Screenshot CDN upload
- └ ML: Train + persist models + Input validation
- └ RAG: Persistent ChromaDB + GDPR document added
- └ Frontend: Remove all mockData + API layer refactor

Week 2 – Stability

- └ Backend: Rate limiting + Usage metering
- └ ML: Parallel feature extraction + Redis scan store
- └ RAG: Conversation memory (Redis session)
- └ Frontend: Real-time progress UI (SSE consumer)

Week 3 – Features

- └ Backend: Webhook support + Health checks
- └ ML: Model versioning + CORS hardening
- └ RAG: Citation confidence + Streaming responses
- └ Frontend: Route discovery modal + AI Assistant wired

Week 4 – Polish

- └ Backend: Email notifications + Security hardening
- └ ML: Feedback loop + Performance benchmarks
- └ RAG: Hallucination prevention + Admin corpus update
- └ Frontend: PDF export + Error boundary + Toast notifications

Week 5-6 – Integration Testing

- └ End-to-end test: URL submission → real-time progress → full report
- └ Multi-page audit test with 5+ pages
- └ AI Assistant: ask follow-up questions about a specific audit
- └ Rate limiting: verify limits enforced
- └ PDF export: verify professional output
- └ Mobile: verify all views on 375px viewport

Week 7 – Performance and Security Audit

- └ Load test: 50 concurrent users
- └ Security: OWASP scan, credential audit
- └ ML accuracy benchmark on held-out test set
- └ RAG legal accuracy benchmark (50 Q&A pairs)
- └ Lighthouse performance audit of Frontend

Week 8 – Production Deployment

- └ Set up production environment (cloud hosting)
- └ Configure all environment variables

- └─ Set up monitoring and alerting
- └─ Soft launch with 10 beta users
- └─ Collect feedback and iterate

Business Model Notes (For Product Decisions)

Tier	Audits/Month	Features	Price (Suggested)
Free	5	Single-page audits only, watermarked PDF	Free
Pro	50	Multi-page audits, full PDF export, AI chat	\$49/month
Enterprise	Unlimited	API access, webhooks, team seats, priority support	\$299/month

The Backend's **Usage Metering feature (Feature 4)** must enforce these tiers. The Backend's **Webhook feature (Feature 5)** enables the Enterprise API access tier.

Definition of "Production Ready" — Master Checklist

Security

- ☐ All credentials rotated and moved to secrets manager
- ☐ No credentials in Git history
- ☐ Rate limiting enforced on all authenticated endpoints
- ☐ CORS restricted on all services
- ☐ API keys not hardcoded to default values

Reliability

- ☐ No synchronous blocking requests over 2 seconds
- ☐ Job queue with retry logic
- ☐ Redis-backed async scan store (ML)
- ☐ Persistent vector database (RAG)
- ☐ Health check endpoint covering all services

Correctness

- ☐ Zero components using mockData in production
- ☐ ML ensemble trained and accuracy documented (target: F1 >= 0.72)
- ☐ GDPR, EU DSA added to legal corpus
- ☐ Citation relevance filtering active (no hallucinated law references)

User Experience

- ☐ Real-time audit progress shown to user
- ☐ Skeleton loading states on all data views
- ☐ AI assistant handles follow-up questions

- ☐ PDF export works
- ☐ Error boundary shows friendly error (not white screen)
- ☐ Mobile responsive

Business

- ☐ Free/Pro/Enterprise tier limits enforced
- ☐ Email notification on audit complete
- ☐ Audit history persistently stored and viewable

Team Document Quick Reference

Team	Document	Priority
Team 1 — Backend (Node.js)	TEAM_1_BACKEND.md	CRITICAL
Team 2 — ML Engine (Python)	TEAM_2_ML.md	HIGH
Team 3 — RAG Legal AI (Python)	TEAM_3_RAG.md	HIGH
Team 4 — Frontend (React)	TEAM_4_FRONTEND.md	HIGH

Immediate Actions (Do These Today)

1. **Rotate ALL credentials in** `Backend/.env` **RIGHT NOW.** Treat them as compromised.
2. **Verify if ML models are saved to disk.** Run `ls -la ML/models/` — if empty, retrain and save.
3. **Verify ChromaDB has data.** Run `ls -la rag/chroma_db/` — if empty, re-run `python ingest.py`.
4. **Share each team document with the relevant developer.** The documents are self-contained.
5. **Set up a Redis instance** (local Docker or Upstash cloud) — both Backend and ML need it.